Implementations reference

Latin American regional preparation. C++17; zero-based indices; array ranges $[l, r)$ unless stated. Choose the model, check bounds, then type. Edition 2026-10-28.

Lookup

1 Implementations **1**
- 1.1 Choose the model first . . . 1
- 1.2 Unweighted graphs . . . 1
- 1.3 Weighted paths and dependencies . . . 2
- 1.4 Connectivity and trees . . . 2
- 1.5 Intervals and monotonicity . . . 3
- 1.6 Dynamic range structures . . . 4
- 1.7 Dynamic programming . . . 4
- 1.8 Small exhaustive search . . . 5
- 1.9 Keep linked values together when sorting . . . 5
- 1.10 Recipes and inventory: validate, then commit . . . 5
- 1.11 Offline queries: activate values in threshold order . 6

2 Specialists **6**
- 2.1 Strings . . . 6
- 2.2 Directed constraints . . . 7
- 2.3 Matching and capacities . . . 8
- 2.4 Tree paths . . . 8
- 2.5 Geometry . . . 9
- 2.6 Optimized DP . . . 9

Implementations (1)

1.1 Choose the model first

Store full state. BFS: unit edges; 0–1 BFS: weights 0/1; Dijkstra: nonnegative; DAG DP: acyclic; Bellman–Ford: general negative edges. Components: undirected connectivity; DSU: additions; Kruskal: spanning forest; SCC: directed mutual reachability.

Choose by asking what one legal move costs and what information determines the future. If every move costs one, BFS minimizes move count even when the graph is implicit (a grid, word changes, or a state machine); materializing all edges is optional. If the state is not just a vertex, use the product state: for example (v, p) for vertex and parity, or $(cell, t)$ only when time cannot be summarized by a static safety bound. Do not collapse two histories merely because they reach the same place if their legal next moves differ. Conversely, avoid carrying history that has no effect on future choices: a remainder often replaces the whole numeric prefix.

For weighted paths, ask whether weights are exactly 0/1, merely nonnegative, or possibly negative. The cheapest-looking queue is not a substitute for this proof: ordinary BFS is wrong as soon as edge costs vary. For dependencies, prove there is no directed cycle before topological DP. For connectivity, distinguish "can reach" from "mutually reachable"; DSU cannot undo deletions and does not represent directed reachability.

For optimization over sequences, write down the compatibility relation between a candidate and its predecessor. If compatible predecessors form a prefix after sorting/compression, a prefix maximum or sum structure can replace a quadratic scan. Check whether equality is allowed before choosing the query endpoint, and process ties as a batch when one equal-key item must not depend on another. For any DP, name the state in a sentence, give its base case, and state why all legal histories mapped to that state have the same continuations. Then count states times transitions; a correct state with a million transitions per state is still not an implementation plan.

When uncertain, build the smallest counterexample to your proposed state: two paths arriving at the same vertex with different parity, two points sharing one coordinate, or a hazard arriving at the same time as you. If the proposed merge loses information needed by the answer, add precisely that information. The worked cases in the pattern casebook show this loop from recognition through invariant, implementation boundary, and a small variant.

1.1.1 Conventions

Zero-based indices; half-open $[l, r)$ ($[L, R]$ becomes $[L-1, R)$). Unreachable weighted distance is `INF=LLONG_MAX/4`; finite values must be smaller. Bound arithmetic; cast before multiplication. Rectangular grids, *hw* fits `int`. Use `finals_core`; reset per case.

Prelude.h

Description: Shared includes, aliases and bounds. Include before the modules.

`<algorithm>, <cassert>, <deque>, <functional>, <limits>, <numeric>, <queue>, <string>, <utility>, <vector>` b9e477, 11 lines

```
namespace finals_core {
using namespace std;
using ll = long long;
using vi = vector<int>;
using vl = vector<ll>;
using Graph = vector<vi>;
using WGraph = vector<vector<pair<int, ll>>>;
constexpr ll INF = numeric_limits<ll>::max() / 4;

} // namespace finals_core
```

1.2 Unweighted graphs

1.2.1 BFS

`bfs(g,sources)` returns edge distances (-1 if unreachable) and parents; sources start at zero. `restore_path(a,t)` returns a vertex path or empty if unreachable. $O(n + m + |S|)$ time, $O(n)$ space.

```
auto a = bfs(g, {source});
auto path = restore_path(a, target);
```

BFS.h

Description: Unweighted BFS and path reconstruction.

`"Prelude.h"` 51289f, 34 lines

```
namespace finals_core {
struct Search {
  vi dist, parent;
};
inline Search bfs(const Graph &g, const vi &sources) {
  int n = int(g.size());
  Search a{vi(n, -1), vi(n, -1)};
  queue<int> q;
  for (int s : sources)
    if (a.dist[s] == -1) {
      a.dist[s] = 0;
      q.push(s);
    }
  while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int v : g[u])
      if (a.dist[v] == -1) {
        a.dist[v] = a.dist[u] + 1;
        a.parent[v] = u;
        q.push(v);
      }
  }
  return a;
}
inline vi restore_path(const Search &a, int target) {
  if (a.dist[target] == -1) return {};
  vi p;
  for (int u = target; u != -1; u = a.parent[u])
    p.push_back(u);
  reverse(p.begin(), p.end());
  return p;
}
} // namespace finals_core
```

1.2.2 Connected components

`components(g)` assumes an undirected graph; returns IDs, representatives, sizes, and count. $c - 1$ edges connect c components. Directed input needs SCC or direction-ignoring traversal. $O(n + m)$ time, $O(n)$ space.

Components.h

Description: Connected components.

`"Prelude.h"` 369a7a, 29 lines

```
namespace finals_core {
struct Components {
  int count = 0;
  vi id, representative, size;
};
inline Components components(const Graph &g) {
  Components a;
  a.id.assign(g.size(), -1);
```

```
  for (int s = 0; s < int(g.size()); ++s)
    if (a.id[s] == -1) {
      int id = a.count++;
      a.representative.push_back(s);
      a.size.push_back(0);
      vi st{s};
      a.id[s] = id;
      while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        ++a.size[id];
        for (int v : g[u])
          if (a.id[v] == -1) {
            a.id[v] = id;
            st.push_back(v);
          }
      }
    }
  return a;
}
} // namespace finals_core
```

1.2.3 Grid BFS

`grid_bfs(board,sources,'#')` uses four neighbors; non-wall cells pass. ID $rw + c$ (decode `id/w,id%w`). Sources in bounds; wall sources ignored. $O(hw)$. Hazards need separate arrival BFS and constrained transitions, often $t < hazard[v]$.

GridBFS.h

Description: Grid BFS.

"Prelude.h", "BFS.h" cb78ea, 34 lines

```
namespace finals_core {
// Four-neighbor grid; every character other than wall is
// traversable.
inline Search grid_bfs(const vector<string> &grid,
                       const vector<pair<int, int>> &sources,
                       char wall = '#') {
  int h = int(grid.size()), w = h ? int(grid[0].size()) : 0;
  Search a{vi(h * w, -1), vi(h * w, -1)};
  queue<int> q;
  for (auto [r, c] : sources) {
    assert(0 <= r && r < h && 0 <= c && c < w);
    if (grid[r][c] != wall && a.dist[r * w + c] == -1) {
      a.dist[r * w + c] = 0;
      q.push(r * w + c);
    }
  }
  const int dr[] = {1, -1, 0, 0}, dc[] = {0, 0, 1, -1};
  while (!q.empty()) {
    int u = q.front();
    q.pop();
    int r = u / w, c = u % w;
    for (int k = 0; k < 4; ++k) {
      int nr = r + dr[k], nc = c + dc[k];
      if (nr < 0 || nr >= h || nc < 0 || nc >= w) continue;
      int v = nr * w + nc;
      if (grid[nr][nc] == wall || a.dist[v] != -1) continue;
      a.dist[v] = a.dist[u] + 1;
      a.parent[v] = u;
      q.push(v);
    }
  }
  return a;
}
} // namespace finals_core
```

1.3 Weighted paths and dependencies

1.3.1 0–1 BFS and Dijkstra

`zero_one_bfs(g,s)` requires weights 0/1; improvements go deque front/back. `dijkstra(g,s)` requires nonnegative weights. Both return `dist,parent`; unreachable is `INF`. Dijkstra skips candidates $\geq$ `INF`. Time: $O(n + m)$ / $O((n + m)\log(n + m + 1))$. Reverse weighted parents through -1; `restore_path` takes unweighted `Search` only.

WeightedPaths.h

Description: Weighted distances: 0-1 BFS and Dijkstra.

"Prelude.h" bfcf5f, 54 lines

```
namespace finals_core {
struct WeightedSearch {
  vl dist;
  vi parent;
};
inline WeightedSearch zero_one_bfs(const WGraph &g,
                                   int source) {
  int n = int(g.size());
  WeightedSearch a{vl(n, INF), vi(n, -1)};
  deque<pair<ll, int>> q;
  a.dist[source] = 0;
  q.push_front({0, source});
  while (!q.empty()) {
    auto [du, u] = q.front();
    q.pop_front();
    if (du != a.dist[u]) continue;
    for (auto [v, w] : g[u]) {
      assert(w == 0 || w == 1);
      if (du + w < a.dist[v]) {
        a.dist[v] = du + w;
        a.parent[v] = u;
        if (w == 0)
          q.push_front({du, v});
        else
          q.push_back({du + 1, v});
      }
    }
  }
  return a;
}
inline WeightedSearch dijkstra(const WGraph &g, int source) {
  int n = int(g.size());
  WeightedSearch a{vl(n, INF), vi(n, -1)};
  using Node = pair<ll, int>;
  priority_queue<Node, vector<Node>, greater<Node>> q;
  a.dist[source] = 0;
  q.push({0, source});
  while (!q.empty()) {
    auto [du, u] = q.top();
    q.pop();
    if (du != a.dist[u]) continue;
    for (auto [v, w] : g[u]) {
      assert(w >= 0);
      if (w >= INF - du) continue;
      if (du + w < a.dist[v]) {
        a.dist[v] = du + w;
        a.parent[v] = u;
        q.push({du + w, v});
      }
    }
  }
  return a;
}
} // namespace finals_core
```

1.3.2 Topological order

`topological_order(g)` returns order or empty on cycle; check size n (empty graph valid). Parallel edges count separately. DAG DP requires acyclicity. $O(n + m)$ time, $O(n)$ space.

TopoSort.h

Description: Topological order.

"Prelude.h" b7ee49, 21 lines

```
namespace finals_core {
// Empty result means a cycle unless the input graph is empty.
inline vi topological_order(const Graph &g) {
  int n = int(g.size());
  vi deg(n), order;
  for (const auto &adj : g)
    for (int v : adj) ++deg[v];
  queue<int> q;
  for (int u = 0; u < n; ++u)
    if (!deg[u]) q.push(u);
  while (!q.empty()) {
    int u = q.front();
    q.pop();
    order.push_back(u);
    for (int v : g[u])
      if (--deg[v] == 0) q.push(v);
  }
  if (int(order.size()) != n) return {};
  return order;
}
} // namespace finals_core
```

1.4 Connectivity and trees

1.4.1 DSU and Kruskal

`DSU(n)`: find representative, unite result, set size; amortized $O(\alpha(n))$, additions only. `kruskal(n,edges)` takes each undirected `Edge{u,v,w}` once; returns forest edges, weight, count. Minimizes edge sum, not source distances; total fits `ll`. $O(m \log m)$.

DSUKruskal.h

Description: Disjoint sets and minimum spanning forests.

"Prelude.h" f3f401, 48 lines

```
namespace finals_core {
struct DSU {
  vi p, sizes;
  explicit DSU(int n) : p(n), sizes(n, 1) {
    iota(p.begin(), p.end(), 0);
  }
  int find(int u) {
    while (u != p[u]) {
      p[u] = p[p[u]];
      u = p[u];
    }
    return u;
  }
  bool unite(int u, int v) {
    u = find(u);
    v = find(v);
    if (u == v) return false;
    if (sizes[u] < sizes[v]) swap(u, v);
    p[v] = u;
```

```
    sizes[u] += sizes[v];
    return true;
  }
  int size(int u) { return sizes[find(u)]; }
};
struct Edge {
  int u, v;
  ll w;
};
struct MST {
  ll weight = 0;
  int components = 0;
  vector<Edge> edges;
};
inline MST kruskal(int n, vector<Edge> edges) {
  sort(edges.begin(), edges.end(),
       [](Edge a, Edge b) { return a.w < b.w; });
  DSU d(n);
  MST a;
  a.components = n;
  for (Edge e : edges)
    if (d.unite(e.u, e.v)) {
      a.weight += e.w;
      a.edges.push_back(e);
      --a.components;
    }
  return a;
}
} // namespace finals_core
```

1.4.2 Tree order and LCA

`root_tree(tree,root)` requires a connected undirected tree; returns parent, depth, parent-first order, subtree sizes. Reverse order supports DP. Cyclic input gives a spanning tree, not validation. Not Euler order: subtree need not be contiguous. $O(n)$. `LCA(info)` provides `jump`, `query`, edge-count `distance`; above-root jump is -1. Preprocess $O(n \log n)$, query $O(\log n)$. Weighted distance needs root sums; cross-component queries invalid.

TreeLCA.h

Description: Tree order, subtree sizes, and LCA.

"Prelude.h" 19e709, 61 lines

```
namespace finals_core {
struct TreeInfo {
  vi parent, depth, order, subtree;
};
inline TreeInfo root_tree(const Graph &tree, int root = 0) {
  int n = int(tree.size());
  assert(n > 0 && 0 <= root && root < n);
  TreeInfo t{vi(n, -1), vi(n), {}, vi(n, 1)};
  t.parent[root] = root;
  t.order.push_back(root);
  for (int i = 0; i < int(t.order.size()); ++i) {
    int u = t.order[i];
    for (int v : tree[u])
      if (t.parent[v] == -1) {
        t.parent[v] = u;
        t.depth[v] = t.depth[u] + 1;
        t.order.push_back(v);
      }
  }
  assert(int(t.order.size()) == n);
  for (int i = n - 1; i > 0; --i) {
    int u = t.order[i];
    t.subtree[t.parent[u]] += t.subtree[u];
  }
  return t;
}
struct LCA {
  vector<vi> up;
  vi depth;
  explicit LCA(const TreeInfo &t) : depth(t.depth) {
    int n = int(depth.size()), levels = 1;
    assert(n > 0);
    while ((1LL << levels) <= n) ++levels;
    up.assign(levels, vi(n));
    up[0] = t.parent;
    for (int k = 1; k < levels; ++k)
      for (int u = 0; u < n; ++u)
        up[k][u] = up[k - 1][up[k - 1][u]];
  }
  int jump(int u, int steps) const {
    if (steps < 0 || steps > depth[u]) return -1;
    for (int k = 0; k < int(up.size()); ++k)
      if ((steps >> k) & 1) u = up[k][u];
    return u;
  }
  int query(int u, int v) const {
    if (depth[u] < depth[v]) swap(u, v);
    u = jump(u, depth[u] - depth[v]);
    if (u == v) return u;
    for (int k = int(up.size()) - 1; k >= 0; --k)
      if (up[k][u] != up[k][v]) {
        u = up[k][u];
        v = up[k][v];
      }
    return up[0][u];
  }
  int distance(int u, int v) const {
    return depth[u] + depth[v] - 2 * depth[query(u, v)];
  }
};
} // namespace finals_core
```

1.5 Intervals and monotonicity

1.5.1 Compression and prefix sums

`compressed_values(a)` sorts/deduplicates; `rank_of(v,x)` returns the lower-bound position (exact only if present); preserves order, not distances. `prefix_sums(a)` has $n+1$ entries; signed $[l, r)$ sum is $P[r] - P[l]$. Prefix build/query $O(n)/O(1)$; compression/lookup $O(n \log n)/O(\log n)$.

CompressionPrefix.h

Description: Coordinate compression and prefix sums.

"Prelude.h" ace528, 19 lines

```
namespace finals_core {
template <class T> vector<T> compressed_values(vector<T> a) {
  sort(a.begin(), a.end());
  a.erase(unique(a.begin(), a.end()), a.end());
  return a;
}
template <class T>
int rank_of(const vector<T> &sorted_unique, const T &x) {
  auto it = lower_bound(sorted_unique.begin(),
                        sorted_unique.end(), x);
  return int(it - sorted_unique.begin());
}
inline vl prefix_sums(const vl &a) {
  vl p(a.size() + 1);
  for (int i = 0; i < int(a.size()); ++i)
    p[i + 1] = p[i] + a[i];
  return p;
}
} // namespace finals_core
```

1.5.2 Union of intervals

`merge_intervals(a)` takes half-open $[l, r)$ intervals with $l \le r$; discards zero lengths and merges touching intervals. Require finite endpoints and no NaNs. $O(n \log n)$; widen endpoints before summing lengths. Used in Regional 2024 J.

IntervalUnion.h

Description: Union of intervals.

"Prelude.h" f002b0, 16 lines

```
namespace finals_core {
template <class T>
vector<pair<T, T>> merge_intervals(vector<pair<T, T>> a) {
  for (auto [l, r] : a) assert(l <= r);
  sort(a.begin(), a.end());
  vector<pair<T, T>> out;
  for (auto [l, r] : a) {
    if (l == r) continue;
    if (out.empty() || out.back().second < l)
      out.push_back({l, r});
    else
      out.back().second = max(out.back().second, r);
  }
  return out;
}
} // namespace finals_core
```

1.5.3 Offline differences

`diff(n+1)`; `difference_add(diff,l,r,x)` adds on $[l, r)$; `materialize_difference(diff)` returns n increments. $O(1)$ update, $O(n)$ materialize; offline only.

DifferenceArray.h

Description: Difference arrays.

"Prelude.h" 77b4e3, 17 lines

```
namespace finals_core {
inline void difference_add(vl &diff, int l, int r, ll value) {
  assert(0 <= l && l <= r && r < int(diff.size()));
  diff[l] += value;
  diff[r] -= value;
}
inline vl materialize_difference(const vl &diff) {
  assert(!diff.empty());
  vl a(diff.size() - 1);
  ll cur = 0;
  for (int i = 0; i < int(a.size()); ++i) {
    cur += diff[i];
    a[i] = cur;
  }
  return a;
}
} // namespace finals_core
```

1.5.4 Sliding window

`count_subarrays_at_most(a,k)` counts nonempty sums $\le k$ for nonnegative entries (zero if $k < 0$). Exact sum: $atMost(k) - atMost(k-1)$. Signed entries need prefix frequencies. $O(n)$ time, $O(1)$ space.

SlidingWindow.h
Description: Sliding window for nonnegative sums.
"Prelude.h" 7093f8, 22 lines

```
namespace finals_core {
// Number of nonempty subarrays with sum <= limit; all a[i]
// must be >= 0.
inline ll count_subarrays_at_most(const vl &a, ll limit) {
  if (limit < 0) return 0;
  int l = 0;
  ll sum = 0, answer = 0;
  for (int r = 0; r < int(a.size()); ++r) {
    assert(a[r] >= 0);
    if (a[r] > limit) {
      l = r + 1;
      sum = 0;
      continue;
    }
    // Shrink before adding: sum+a[r] may overflow ll.
    while (sum > limit - a[r]) sum -= a[l++];
    sum += a[r];
    answer += r - l + 1;
  }
  return answer;
}
} // namespace finals_core
```

1.5.5 Binary search on the answer

`first_true(lo,hi,ok)` searches false-then-true on nonnegative $[lo, hi)$; skips hi, returns it if none succeeds. Prove monotonicity and sentinel feasibility. $O(\log(hi - lo + 1)\,\mathrm{cost}(ok))$.

BinarySearch.h
Description: Monotone binary search.
"Prelude.h" 872296, 14 lines

```
namespace finals_core {
// Predicate is false, ..., false, true, ..., true on [lo,hi).
template <class Pred> ll first_true(ll lo, ll hi, Pred ok) {
  assert(0 <= lo && lo <= hi);
  while (lo < hi) {
    ll mid = lo + (hi - lo) / 2;
    if (ok(mid))
      hi = mid;
    else
      lo = mid + 1;
  }
  return lo; // Original hi if every tested position is false.
}
} // namespace finals_core
```

1.6 Dynamic range structures

1.6.1 Fenwick tree

`Fenwick(n)`: `add(i,d)`, `prefix(r)` on $[0, r)$, `sum(l,r)`. Assignment uses a delta. $O(\log n)$ operations, $O(n)$ space. `lower_bound(k)` requires $k > 0$ and nonnegative entries; returns the zero-based index of the first element whose inclusive prefix reaches k, or n if absent. Rank and range arguments elsewhere use zero-based indices; Fenwick updates map those indices to one-based internal storage, while `prefix(r)` still queries $[0, r)$.

Fenwick.h
Description: Fenwick sums and order statistics.
"Prelude.h" 6a5916, 37 lines

```
namespace finals_core {
struct Fenwick {
  int n;
  vl bit;
  explicit Fenwick(int n_) : n(n_), bit(n_ + 1) {}
  void add(int i, ll delta) {
    assert(0 <= i && i < n);
    for (++i; i <= n; i += i & -i) bit[i] += delta;
  }
  ll prefix(int r) const {
    assert(0 <= r && r <= n);
    ll answer = 0;
    for (; r; r -= r & -r) answer += bit[r];
    return answer;
  }
  ll sum(int l, int r) const {
    assert(0 <= l && l <= r && r <= n);
    return prefix(r) - prefix(l);
  }
  // Positive, one-based target; all represented values must be
  // nonnegative.
  int lower_bound(ll target) const {
    assert(target > 0);
    int pos = 0, step = 1;
    ll accumulated = 0;
    while (step <= n / 2) step *= 2;
    for (; step; step /= 2) {
      int next = pos + step;
      if (next <= n && bit[next] < target - accumulated) {
        accumulated += bit[next];
        pos = next;
      }
    }
    return pos; // n means no prefix reaches target.
  }
};
} // namespace finals_core
```

1.6.2 Iterative segment tree

`SegTree<T,Op>(a,identity,op)` supports point set and ordered query on $[l, r)$. `Op` associative with two-sided identity; need not commute. Empty query returns identity. Build/space $O(n)$, operations $O(\log n)$.

SegmentTree.h
Description: Iterative segment tree.
"Prelude.h" 3f3746, 36 lines

```
namespace finals_core {
struct Sum {
  ll operator()(ll a, ll b) const { return a + b; }
};
struct Minimum {
  ll operator()(ll a, ll b) const { return min(a, b); }
};
template <class T, class Op> struct SegTree {
  int n, base = 1;
  T identity;
  Op op;
  vector<T> t;
  SegTree(const vector<T> &a, T identity_, Op op_ = Op())
      : n(int(a.size())), identity(identity_), op(op_) {
    while (base < n) base *= 2;
    t.assign(2 * base, identity);
    for (int i = 0; i < n; ++i) t[base + i] = a[i];
    for (int i = base - 1; i; --i)
      t[i] = op(t[2 * i], t[2 * i + 1]);
  }
  void set(int i, T value) {
    assert(0 <= i && i < n);
    t[i += base] = value;
    while (i /= 2) t[i] = op(t[2 * i], t[2 * i + 1]);
  }
  T query(int l, int r) const {
    assert(0 <= l && l <= r && r <= n);
    T left = identity, right = identity;
    for (l += base, r += base; l < r; l /= 2, r /= 2) {
      if (l & 1) left = op(left, t[l++]);
      if (r & 1) right = op(t[--r], right);
    }
    return op(left, right);
  }
};
} // namespace finals_core
```

1.7 Dynamic programming

Define what one state means. Equal states must have the same legal futures. Write base cases, transitions and evaluation order; distinguish unreachable from zero. Count states times transitions before coding. Tiny brute force checks whether forgotten history changes an answer.

1.7.1 Knapsack

`knapsack01(w,v,C)` maximizes value at most C, allows nothing and nonnegative weights (zero included); scan downward. Unbounded requires positive weights; scan upward. Both $O(nC)$ time/$O(C)$ space. Positive-value zero-weight unbounded item gives infinity. Exact base: $D[0] = 0$, others $-\infty$; at-most base: all zero.

Knapsack.h
Description: Capacity DP: 0-1 and unbounded knapsack.
"Prelude.h" 4b4e7e, 26 lines

```
namespace finals_core {
// Max value with total weight <= capacity. Taking no items is
// allowed.
inline ll knapsack01(const vi &weight, const vl &value,
                     int capacity) {
  assert(weight.size() == value.size() && capacity >= 0);
  vl dp(capacity + 1);
  for (int i = 0; i < int(weight.size()); ++i) {
    assert(weight[i] >= 0);
    for (int c = capacity; c >= weight[i]; --c)
      dp[c] = max(dp[c], dp[c - weight[i]] + value[i]);
  }
  return dp[capacity];
}
inline ll knapsack_unbounded(const vi &weight, const vl &value,
                             int capacity) {
  assert(weight.size() == value.size() && capacity >= 0);
  vl dp(capacity + 1);
  for (int i = 0; i < int(weight.size()); ++i) {
    assert(weight[i] > 0);
    for (int c = weight[i]; c <= capacity; ++c)
      dp[c] = max(dp[c], dp[c - weight[i]] + value[i]);
  }
  return dp[capacity];
}
} // namespace finals_core
```

1.7.2 Counting and LIS

Coin-outer/sum-ascending counts combinations; sum-outer counts sequences. `lis_indices(a)` returns one strict LIS as indices in $O(n \log n)$; nondecreasing uses `upper_bound` instead of `lower_bound`.

LIS.h

Description: Strict LIS with reconstruction.

"Prelude.h" 0b1033, 24 lines

```
namespace finals_core {
template <class T> vi lis_indices(const vector<T> &a) {
  vector<T> tails;
  vi ending, parent(a.size(), -1);
  for (int i = 0; i < int(a.size()); ++i) {
    auto it = lower_bound(tails.begin(), tails.end(), a[i]);
    int p = int(it - tails.begin());
    if (p > 0) parent[i] = ending[p - 1];
    if (p == int(tails.size())) {
      tails.push_back(a[i]);
      ending.push_back(i);
    } else {
      tails[p] = a[i];
      ending[p] = i;
    }
  }
  vi result;
  if (!ending.empty())
    for (int i = ending.back(); i != -1; i = parent[i])
      result.push_back(i);
  reverse(result.begin(), result.end());
  return result;
}
} // namespace finals_core
```

1.7.3 Digit patterns

`finals_dp::complete_digits(pattern,m)` returns lexicographically smallest divisible completion or `nullopt`; nonempty digits/`?`, $m > 0$. Leading zero forbidden by default, including one digit; pass `false` to allow. $O(10Lm)$ time, $O(Lm)$ memory. Counting to N needs a separate tight/started DP.

DigitDP.h

Description: Smallest divisible decimal-pattern completion. O(L*m*10) time, O(L*m) memory.

<optional>, <stdexcept>, <string>, <vector> de132d, 52 lines

```
namespace finals_dp {
using namespace std;
// Nonempty pattern of digits and '?', positive modulus;
// indices fit int. smallest valid completion divisible by
// modulus, or nullopt. With forbid_leading_zero, even a
// one-digit zero is forbidden. Time O(length * modulus * 10),
// memory O(length * modulus).
inline optional<string>
complete_digits(const string &pattern, int modulus,
                bool forbid_leading_zero = true) {
  if (pattern.empty() || modulus <= 0)
    throw invalid_argument(
        "nonempty pattern and positive modulus required");
  for (char c : pattern)
    if (c != '?' && (c < '0' || c > '9'))
      throw invalid_argument(
          "pattern must contain digits or question marks");
  int n = int(pattern.size());
  vector<vector<unsigned char>> can(
      n + 1, vector<unsigned char>(modulus, 0));
  can[n][0] = 1;
  auto allowed = [&](int pos, int digit) {
    return !(forbid_leading_zero && pos == 0 && digit == 0) &&
           (pattern[pos] == '?' ||
            pattern[pos] == char('0' + digit));
  };
  for (int pos = n - 1; pos >= 0; --pos)
    for (int rem = 0; rem < modulus; ++rem)
      for (int d = 0; d <= 9; ++d)
        if (allowed(pos, d)) {
          int next = int((10LL * rem + d) % modulus);
          if (can[pos + 1][next]) {
            can[pos][rem] = 1;
            break;
          }
        }
  if (!can[0][0]) return nullopt;
  string answer;
  int rem = 0;
  for (int pos = 0; pos < n; ++pos)
    for (int d = 0; d <= 9; ++d)
      if (allowed(pos, d)) {
        int next = int((10LL * rem + d) % modulus);
        if (can[pos + 1][next]) {
          answer += char('0' + d);
          rem = next;
          break;
        }
      }
  return answer;
}
} // namespace finals_dp
```

1.8 Small exhaustive search

For k binary choices enumerate masks $0 \le s < 2^k$. Minimize `__builtin_popcount(s)` when minimizing selected actions. Masters Mexico 2022 F has nine cells: precompute each click's 9-bit toggle mask; XOR chosen masks into the starting board and compare with the target. Clicks commute and pairs cancel, so 512 subsets suffice. $O(k2^k)$; use unsigned shifts with k below the type width.

1.9 Keep linked values together when sorting

Recognize: each object has several fields, but only one field chooses the order. Sorting each array independently destroys the objects. Prefer one record per object when you control the representation:

```
struct Row { int key, price, quantity, id; };
vector<Row> rows;
stable_sort(rows.begin(), rows.end(),
    [](const Row& a, const Row& b) {
        return a.key < b.key;
    });
```

If the input already uses parallel arrays, sort indices once and apply the same permutation to every array. With keys $[4, 1, 4]$, prices $[40, 10, 41]$ and quantities $[2, 9, 3]$, the permutation $[1, 0, 2]$ preserves both equal-key records in input order. Sorting prices separately would lose that association.

Prelude.h

Description: Shared includes, aliases and bounds. Include before the modules.

<algorithm>, <numeric>, <stdexcept>, <utility>, <vector> 9fc94d, 6 lines

```
using namespace std;
namespace finals_practical {
using ll = long long;

} // namespace finals_practical
```

LinkedSort.h

Description: Sort linked records without losing associations.

"Prelude.h" 45b284, 19 lines

```
namespace finals_practical {
// Ascending key, original order breaks ties. O(n log n).
template <class T>
vector<int> sorted_indices(const vector<T> &key) {
  vector<int> p(key.size());
  iota(p.begin(), p.end(), 0);
  stable_sort(p.begin(), p.end(),
              [&](int a, int b) { return key[a] < key[b]; });
  return p;
}
// p must be a permutation of [0,a.size()).
template <class T>
vector<T> reordered(const vector<T> &a, const vector<int> &p) {
  vector<T> out;
  out.reserve(p.size());
  for (int i : p) out.push_back(a[i]);
  return out;
}
} // namespace finals_practical
```

```
auto p = finals_practical::sorted_indices(arr1);
arr1 = finals_practical::reordered(arr1, p);
arr2 = finals_practical::reordered(arr2, p);
arr3 = finals_practical::reordered(arr3, p);
```

All arrays must have the same length. This is linked sorting, not coordinate compression: compression replaces distinct values with ranks. For a secondary key, compare `tie(a.key,a.price)`; for descending primary order reverse only that comparison. Never use <= in a sort comparator. Do not compare by subtraction, which can overflow. Stable input-order ties are a choice, not the correct tie rule for every DP; strict dominance needs the batching described in Fundraising.

1.10 Recipes and inventory: validate, then commit

Recognize: an operation consumes several shared resources and must succeed completely or leave the state unchanged. Accumulate its requirements first, check every limit, then subtract once. This pattern applies to recipes, warehouse orders, machine resources, and bundles of tickets.

If stock is $[5, 4]$, recipe A needs $(0, 2), (1, 1)$ and recipe B needs $(0, 2)$, order $[A, B]$ consumes $[4, 1]$, leaving $[1, 3]$. A second order $[A]$ fails and must leave $[1, 3]$ unchanged. Repeated dishes and repeated ingredient IDs within a recipe both add to the requirement; a set would be incorrect.

AtomicInventory.h

Description: Validate an entire order before consuming inventory.

"Prelude.h" c8b7c0, 36 lines

```
namespace finals_practical {
using Recipe = vector<pair<int, ll>>;
struct Inventory {
  vector<ll> stock, need;
  vector<int> touched;
  // All stock and quantities nonnegative. IDs in range.
  explicit Inventory(vector<ll> initial)
      : stock(std::move(initial)), need(stock.size(), 0) {}
  // Duplicate ingredients/dishes accumulate. Failure is
  // atomic. O(order length + expanded ingredient entries +
  // touched).
  bool consume(const vector<Recipe> &recipes,
               const vector<int> &order) {
    bool ok = true;
    for (int dish : order) {
      for (auto [id, q] : recipes[dish]) {
        if (q == 0) continue;
        if (need[id] == 0) touched.push_back(id);
        // Subtract first: no overflowing need + q.
        if (q > stock[id] - need[id]) {
          ok = false;
          break;
        }
        need[id] += q;
      }
      if (!ok) break;
    }
    for (int id : touched) {
      if (ok) stock[id] -= need[id];
      need[id] = 0;
    }
    touched.clear();
    return ok;
  }
};
} // namespace finals_practical
```

```
finals_practical::Inventory inv(stock);
int accepted = 0;
for (const auto& order : orders) {
    if (!inv.consume(recipes, order)) break;
    ++accepted;
}
```

This loop answers the number of consecutive fulfillable orders, stopping at the first failure, as in the supplied example. If the problem says to reject an impossible order and continue, use `continue` instead of `break`. If it lets you choose or reorder orders to maximize acceptance, this simulation does not solve the optimization problem.

Bounds: use 64-bit stock and quantities. Each is nonnegative; check `q > stock[id]-need[id]` before adding, so even an oversized request cannot overflow. All recipe and ingredient IDs must be valid; convert one-based input exactly once. Zero quantities and empty orders are supported. Negative quantities would invalidate the early-failure argument.

The touched-index list clears only ingredients examined in this order, rather than allocating and scanning an N-entry array every time. Initialization is $O(N)$; each call costs its dish count plus expanded ingredient entries and touched indices. Repeated large recipes still cost work per occurrence. Orders may be read and processed one at a time to avoid storing them all. When an early order fails, still read remaining input if a surrounding multi-test-case format requires it.

1.11 Offline queries: activate values in threshold order

Recognize: a fixed array, many interval queries, and each query counts values at least a threshold. Queries can be answered out of input order, then restored using their original indices. Sort values and queries by decreasing threshold. Before answering a query, activate every array position whose value is at least its threshold. A Fenwick tree stores one at active positions, so a range sum gives the count.

OfflineThreshold.h

Description: Offline threshold queries on static ranges.

"Prelude.h", "LinkedSort.h" 1fcb1c, 35 lines

```
namespace finals_practical {
struct ThresholdQuery {
  int l, r;
  ll minimum;
};
// Count a[i]>=minimum for l<=i<r; valid half-open ranges.
// O((n+q) log(n+q)) time, O(n+q) space; no updates.
inline vector<int>
threshold_counts(const vector<ll> &a,
                 const vector<ThresholdQuery> &queries) {
  int n = a.size(), q = queries.size();
  auto pos = sorted_indices(a);
  reverse(pos.begin(), pos.end());
  vector<int> ids(q), bit(n + 1), ans(q);
  iota(ids.begin(), ids.end(), 0);
  sort(ids.begin(), ids.end(), [&](int x, int y) {
    return queries[x].minimum > queries[y].minimum;
  });
  auto prefix = [&](int end) {
    int sum = 0;
    for (; end; end -= end & -end) sum += bit[end];
    return sum;
  };
  int active = 0;
  for (int id : ids) {
    auto [l, r, minimum] = queries[id];
    while (active < n && a[pos[active]] >= minimum) {
      for (int i = pos[active++] + 1; i <= n; i += i & -i)
        ++bit[i];
    }
    ans[id] = prefix(r) - prefix(l);
  }
  return ans;
}
} // namespace finals_practical
```

For $a = [5, 2, 7, 4]$ and query $[1, 4)$ with threshold 5, positions 0 and 2 are active; only position 2 lies in the interval, so the answer is 1. At threshold 4, activate position 3 too and the answer becomes 2. Equal values must be activated before an inclusive query. For strictly greater values, change the activation comparison to >.

This preserves the original array indices; sorting the values and then querying their sorted positions would answer a different question. All ranges must satisfy $0 \le l \le r \le n$. The array is static: interleaved updates require a different method. After Euler flattening, this same helper answers threshold queries on subtrees. The output order matches the query input order even though evaluation order changes.

Specialists (2)

Prelude required below; GNU C++17 for SCC/2-SAT. Rename colliding globals/types when combining snippets. Recursive DFS needs sufficient stack; upstream test status does not validate your reduction.

```
#include <bits/stdc++.h>
using namespace std;
#define rep(i,a,b) for(int i=(a);i<(b);++i)
#define all(x) begin(x),end(x)
#define sz(x) int((x).size())
using ll=long long;
using vi=vector<int>;
using pii=pair<int,int>;
```

2.1 Strings

2.1.1 KMP: one pattern, borders

Recognize a single exact pattern searched in a long text, especially when overlaps or borders matter. Prefer this to a general automaton when matches are independent; if many patterns arrive together, use Aho–Corasick instead.

`match(text,pat)`: zero-based starts, overlaps included; nonempty pattern, no nul bytes. Repetition: $p = n - \texttt{pi(s).back()}$, $n \bmod p = 0$.

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.

Time: $\mathcal{O}(n)$

d4375c, 16 lines

```
vi pi(const string& s) {
  vi p(sz(s));
  rep(i,1,sz(s)) {
    int g = p[i-1];
    while (g && s[i] != s[g]) g = p[g-1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}
```

```
vi match(const string& s, const string& pat) {
  vi p = pi(pat + '\0' + s), res;
  rep(i,sz(p)-sz(s),sz(p))
    if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
  return res;
}
```

2.1.2 Z: prefix matches

Recognize repeated comparisons against prefixes, such as matching a pattern after concatenating it with a separator. Use it when the answer needs every suffix's prefix-match length; use KMP for one streaming pattern instead. `Z(s)[i]`: common-prefix length with suffix i, `z[0]=0`. For pattern–separator–text, separator must be absent from both inputs.

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)
Time: $\mathcal{O}(n)$ ee09e2, 12 lines

```
vi Z(const string& S) {
  vi z(sz(S));
  int l = -1, r = -1;
  rep(i,1,sz(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
      z[i]++;
    if (i + z[i] > r)
      l = i, r = i + z[i];
  }
  return z;
}
```

2.1.3 Aho–Corasick: many patterns

Recognize many fixed patterns queried against one or more texts. Build once, then scan each text; for one pattern KMP is smaller and easier to adapt. `AhoCorasick ac(patterns);` `ac.find(text)`: longest ending pattern index, -1 absent. Uppercase A–Z; change alphabet first; no empty patterns.

AhoCorasick.h

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(−, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries.
Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$. f35677, 66 lines

```
struct AhoCorasick {
  enum {alpha = 26, first = 'A'}; // change this!
  struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1, end = -1, nmatches = 0;
    Node(int v) { memset(next, v, sizeof(next)); }
  };
  vector<Node> N;
  vi backp;
  void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int& m = N[n].next[c - first];
      if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
      else n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vector<string>& pat) : N(1, -1) {
    rep(i,0,sz(pat)) insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);

    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
      int n = q.front(), prev = N[n].back;
      rep(i,0,alpha) {
        int &ed = N[n].next[i], y = N[prev].next[i];
        if (ed == -1) ed = y;
        else {
          N[ed].back = y;
          (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
            = N[y].end;
          N[ed].nmatches += N[y].nmatches;
          q.push(ed);
        }
      }
    }
  }
  vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  }
  vector<vi> findAll(vector<string>& pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i,0,sz(word)) {
      int ind = r[i];
      while (ind != -1) {
        res[i - sz(pat[ind]) + 1].push_back(ind);
        ind = backp[ind];
      }
    }
    return res;
  }
};
```

2.2 Directed constraints

2.2.1 SCC: strong components

Recognize mutual reachability or a directed graph that should collapse into groups of mutually reachable vertices. The condensation is a DAG; use reachability or topological DP there instead of repeatedly exploring cycles in the original. `scc(g,callback)` fills `comp[v]`,`ncomps`; edges between components go toward smaller IDs. Copy callback vectors if storing; storage is reused.

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.
Usage: `scc(graph, [&](vi& v) { ... })` visits all components in reverse topological order. `comp[i]` holds the component index of a node (a component only has edges to components with lower index). `ncomps` will contain the number of components.
Time: $\mathcal{O}(E+V)$ 76b5c9, 24 lines

```
vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
  int low = val[j] = ++Time, x; z.push_back(j);
  for (auto e : g[j]) if (comp[e] < 0)
    low = min(low, val[e] ?: dfs(e,g,f));

  if (low == val[j]) {
    do {
      x = z.back(); z.pop_back();
      comp[x] = ncomps;
      cont.push_back(x);
    } while (x != j);
    f(cont); cont.clear();
    ncomps++;
  }
  return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
  int n = sz(g);
  val.assign(n, 0); comp.assign(n, -1);
  Time = ncomps = 0;
  rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
}
```

2.2.2 2-SAT: Boolean clauses

Recognize Boolean choices with clauses involving at most two literals, such as "not both" or "at least one of these two." If constraints include capacities or general sums, use a different model; a satisfiable result gives one assignment, not an optimum. `TwoSat ts(n); ts.either(a,b)` adds $a \vee b$; negation is `~x`, never `!x`. Read `values` only if `solve()` succeeds.

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&\ldots$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).

Usage: `TwoSat ts(number of boolean variables);`
`ts.either(0, ~3); // Var 0 is true or var 3 is false`
`ts.setValue(2); // Var 2 is true`
`ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true`
`ts.solve(); // Returns true iff it is solvable`
`ts.values[0..N-1] holds the assigned values to the vars`
Time: $\mathcal{O}(N+E)$, where N is the number of boolean variables, and E is the number of clauses.

5f9706, 56 lines

```
struct TwoSat {
  int N;
  vector<vi> gr;
  vi values; // 0 = false, 1 = true

  TwoSat(int n = 0) : N(n), gr(2*n) {}

  int addVar() { // (optional)
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }

  void either(int f, int j) {
    f = max(2*f, -1-2*f);
    j = max(2*j, -1-2*j);
    gr[f].push_back(j^1);
    gr[j].push_back(f^1);
  }
  void setValue(int x) { either(x, x); }

  void atMostOne(const vi& li) { // (optional)
    if (sz(li) <= 1) return;
    int cur = ~li[0];
    rep(i,2,sz(li)) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }

  vi val, comp, z; int time = 0;
  int dfs(int i) {
    int low = val[i] = ++time, x; z.push_back(i);
    for(int e : gr[i]) if (!comp[e])
      low = min(low, val[e] ?: dfs(e));
    if (low == val[i]) do {
      x = z.back(); z.pop_back();
      comp[x] = low;
      if (values[x>>1] == -1)
        values[x>>1] = x&1;
    } while (x != i);
    return val[i] = low;
  }

  bool solve() {
    values.assign(N, -1);
    val.assign(2*N, 0); comp = val;
    rep(i,0,2*N) if (!comp[i]) dfs(i);
    rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
    return 1;
  }
};
```

2.3 Matching and capacities

2.3.1 Hopcroft–Karp: bipartite matching

Recognize one-to-one assignment where each left item can use selected right items and each right item is used at most once. Weighted preferences need a weighted matching method; shared seat quotas need a capacity network. `g[left]` lists right indices; initialize `vi r(rightSize,-1)`. `hopcroftKarp(g,r)` returns size, writes `r[right]=left`; unweighted only.

HopcroftKarp.h
Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and r should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $r[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Time: $\mathcal{O}\left(E\sqrt{V}\right)$

731cfb, 20 lines

```
int hopcroftKarp(vector<vi>& g, vi& r) {
  int n = sz(g), res = 0;
  vi l(n, -1), q(n), d(n);
  auto dfs = [&](auto f, int u) -> bool {
    int t = exchange(d[u], 0) + 1;
    for (int v : g[u])
      if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
        return l[u] = v, r[v] = u, 1;
    return 0;
  };
  for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
    rep(i,0,n) if (l[i] == -1) q[t++] = i, d[i] = 1;
    rep(i,0,t) for (int v : g[q[i]]) {
      if (r[v] == -1) f = 1;
      else if (!d[r[v]]) d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
    }
    if (!f) return res;
    rep(i,0,n) if (l[i] == -1) res += dfs(dfs, i);
  }
}
```

2.3.2 Dinic: flow, residual cut

Recognize capacities, routing, or a feasibility model with a source, sink, and conservation at intermediate nodes. Matching is a special case; keep the flow model when quotas, shared bottlenecks, or cut explanations matter. `Dinic f(n);` `f.addEdge(u,v,cap);` `f.calc(s,t)` returns additional flow. Nonnegative capacities, $s \neq t$, no self-loops; totals fit `ll`; cut via `leftOfMinCut`.

Dinic.h
Description: Flow algorithm with complexity $O(VE\log U)$ where $U = \max|\text{cap}|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.

d7f0f1, 42 lines

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster for random data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

2.4 Tree paths

2.4.1 Binary lifting: jumps, LCA

Recognize many ancestor, k-th ancestor, or lowest-common-ancestor queries on a static rooted tree. Updates to parent edges invalidate the table; for changing trees use a dynamic-tree structure instead. `up=treeJump(parent);` `lca(up,depth,u,v);` `parent[root]=root.` `jmp(up,v,k)` requires supported bits; supply valid parents and depths.

BinaryLifting.h
Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.
Time: construction $\mathcal{O}(N\log N)$, queries $\mathcal{O}(\log N)$

bfce85, 25 lines

```
vector<vi> treeJump(vi& P){
  int on = 1, d = 1;
  while(on < sz(P)) on *= 2, d++;
  vector<vi> jmp(d, P);
  rep(i,1,d) rep(j,0,sz(P))
    jmp[i][j] = jmp[i-1][jmp[i-1][j]];
  return jmp;
}

int jmp(vector<vi>& tbl, int nod, int steps){
  rep(i,0,sz(tbl))
    if(steps&(1<<i)) nod = tbl[i][nod];
  return nod;
}

int lca(vector<vi>& tbl, vi& depth, int a, int b) {
  if (depth[a] < depth[b]) swap(a, b);
  a = jmp(tbl, a, depth[a] - depth[b]);
  if (a == b) return a;
```

```
  for (int i = sz(tbl); i--;) {
    int c = tbl[i][a], d = tbl[i][b];
    if (c != d) a = c, b = d;
  }
  return tbl[0][a];
}
```

2.5 Geometry

2.5.1 Point: vector predicates

Recognize orientation, signed area, and projection tests that can be written as dot or cross products. Use integer coordinates for exact predicates when possible; constructed intersection coordinates need separate precision care. `a.cross(b,c)` is $(b-a)\times(c-a)$; positive means left turn. Bound subtraction/products first ($8B^2$ for $|x|,|y|\le B$); integer unit vectors truncate.

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

47ec0a, 28 lines

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
};
```

2.5.2 On segment: closed membership

Recognize whether a point lies on a closed segment, including its endpoints. This is a predicate, not a nearest-point routine; floating coordinates need an explicit tolerance model. `onSegment(s,e,p)` includes endpoints/degenerate segments. Exact equality: floating points need a distance/tolerance alternative.

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use `(segDist(s,e,p)<=epsilon)` instead when using Point<double>.

"Point.h" c597e8, 3 lines

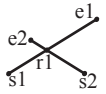
```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

2.5.3 Segment intersection: constructed coordinates

Recognize intersection questions where the actual point or overlap endpoints are required. If the task only asks whether segments cross, exact orientation tests avoid division; this routine constructs coordinates and can truncate. `segInter(a,b,c,d)` returns 0/1/2 points: none/point/overlap endpoints. Integer output truncates fractional intersections; intermediate triple products can overflow.

SegmentIntersection.h

Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.



Usage:
```
vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;
```

"Point.h", "OnSegment.h" 9d57f2, 13 lines

```
template<class P> vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

2.5.4 Convex hull: boundary vertices

Recognize the smallest convex boundary containing a point set, often as a preprocessing step for diameter or containment. Collinear boundary points may be discarded, so retain the original set if those points carry meaning. `convexHull(points)` returns CCW hull, discarding intermediate collinear points. Check hull size before subsequent operations; empty/collinear inputs are degenerate.

ConvexHull.h

Description:
Returns a vector of the points of the convex hull in counterclockwise order. Points on the edge of the hull between two other points are not considered part of the hull.



Time: $\mathcal{O}(n \log n)$

"Point.h" 310954, 13 lines

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

2.5.5 Polygon: signed doubled area

Recognize area of a simple polygon supplied in boundary order; the shoelace sum keeps half-integer answers exact. It does not validate simplicity or sort unordered points into a boundary. `polygonArea2(poly)` requires nonempty boundary order; positive CCW. Bound accumulated cross products; retain doubled area for exact half-unit output.

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h" f12300, 6 lines

```
template<class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
  return a;
}
```

2.6 Optimized DP

2.6.1 Line container: maximum of lines

Recognize a DP transition that asks for the maximum of many affine functions at changing query coordinates. If slopes or query order are constrained, a monotone hull may be simpler; verify arithmetic bounds either way. `LineContainer lc;` `lc.add(k,b);` `lc.query(x)`: maximum $kx+b$; query nonempty. Arbitrary order; products/breakpoint differences fit `ll`; negate carefully for minima.

LineContainer.h

Description: Container where you can add lines of the form kx+m, and query maximum values at points x. Useful for dynamic programming ("convex hull trick").

Time: $\mathcal{O}(\log N)$

8ec1c7, 30 lines

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
```

```
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

2.6.2 Matrix: small-state transition powers

Recognize a linear transition repeated a huge number of times over a small state vector, such as counting walks of fixed length. For a few steps, direct iteration avoids matrix setup; confirm the row/column convention carefully. `Matrix<T,N> A;` `(A^k)*state`; rows destinations, columns sources, $k \geq 0$. Initially zero; exponent zero is identity. Supply modular arithmetic if required.

Matrix.h

Description: Basic operations on square matrices.
Usage: `Matrix<int, 3> A;`
`A.d = {{{1,2,3}}, {{4,5,6}}, {{7,8,9}}};`
`array<int, 3> vec = {1,2,3};`
`vec = (A^N) * vec;`

4da5a2, 26 lines

```
template<class T, int N> struct Matrix {
  typedef Matrix M;
  array<array<T, N>, N> d{};
  M operator*(const M& m) const {
    M a;
    rep(i,0,N) rep(j,0,N)
      rep(k,0,N) a.d[i][k] += d[i][j] * m.d[j][k];
    return a;
  }
  array<T, N> operator*(const array<T, N>& vec) const {
    array<T, N> ret{};
    rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
    return ret;
  }
  M operator^(ll p) const {
    assert(p >= 0);
    M a, b(*this);
    rep(i,0,N) a.d[i][i] = 1;
    while (p) {
      if (p&1) a = a*b;
      b = b*b;
      p >>= 1;
    }
    return a;
  }
};
```